

Product Requirements Document: CipherFlow

Full title: Payload-Free Classification of Encrypted Network Traffic Using Self-Supervised Flow-Shape Tokenisation

Field	Value
Document status	As-built, v1.0
Last updated	2026-10-07
Owners	Rakshit Sinha; Suyash Dabholkar
Institution	Vellore Institute of Technology
Repository	Arvoxis/CipherFlow (private, pre-filing)
Maturity	TRL 4, validated in a laboratory environment
Related documents	patent_draft.md (specification), CipherFlow_IDF.docx (disclosure form), RESULTS.md (measured results)

Nature of this document. CipherFlow is already built and validated, so this is a retrospective PRD. Requirements are stated as they were actually met, and every number in the success-metrics section traces to a run recorded in [artifacts/run_manifest.json](#). Where a requirement was relaxed or a result went against the project, that is recorded rather than quietly rewritten.

1. Problem Statement

Transport encryption has removed the information that traffic-inspection equipment was built to read. TLS 1.3 encrypts the certificate exchange and collapses the handshake; QUIC carries almost everything inside an encrypted packet. Deep packet inspection matches patterns against payload bytes, so against either protocol it returns nothing useful.

An operator who still needs to know what is crossing the network has three unsatisfactory options, plus a fourth problem that cuts across all of them.

Terminate TLS at a middlebox. Requires installing a certificate authority on every endpoint and creates a decryption point that is itself a high-value target. Many deployments cannot do this at all.

Learn over whatever bytes remain visible. Degrades as encryption coverage grows and is defeated cheaply by byte-level obfuscation or re-encoding.

Compute aggregate per-flow statistics. Survives encryption, but discards the order in which packets arrived, so two flows with identical means and variances but entirely different request-response structure become indistinguishable. Also demands a large hand-labelled corpus that goes stale whenever an

application changes behaviour.

And an adversary who cannot break the encryption can still reshape the flow. Padding packets towards the MTU and jittering transmission times costs nothing, needs no protocol change, and is already available in commodity tooling. A classifier keyed to exact sizes or exact gaps collapses when this is done.

2. Users and Use Cases

User	Need	How CipherFlow serves it
Network security analyst	Identify applications and detect command-and-control traffic on an encrypted segment	Classifies from a tap or span port with no decryption and no endpoint cooperation
SOC engineer	Detect malicious flows that survive an evasion attempt	Evasion-augmented model retains accuracy under packet padding and timing jitter
Network operator	Traffic visibility without acquiring a plaintext choke point	Payload never enters the pipeline, so the apparatus cannot leak it or be compelled to produce it
Researcher	Reproduce published results and extend the method	One-command pipeline, provenance manifest, CI, synthetic generator needing no downloads
Patent examiner or attorney	Verify the invention was reduced to practice	Live demo, dated run manifests, results tied to exact code revisions

Primary use case. An apparatus at a monitoring position observes frames, assembles bidirectional flows, and emits a class label plus confidence per flow, having read no payload.

Secondary use case. A researcher evaluates how much labelled traffic a new deployment needs before the classifier is useful, by running the label-efficiency sweep.

3. Goals and Non-Goals

Goals

#	Goal
G1	Classify encrypted flows without decrypting them and without reading or retaining payload
G2	Preserve temporal packet order, which aggregate-statistic methods discard
G3	Reduce the labelled traffic a deployment must produce, by pre-training on unlabelled traffic
G4	Retain accuracy when an adversary pads packet sizes or jitters packet timings

#	Goal
G5	Keep per-flow state small and independent of the payload volume the flow carries
G6	Ship a deployable end-to-end pipeline: capture, tokenise, pre-train, fine-tune, evaluate, serve
G7	Make every result reproducible from an exact code revision, config hash and seed

Non-Goals

#	Non-goal	Why
N1	Decrypting traffic or terminating TLS	Defeats the premise; any payload read breaks the core patent claim
N2	Beating aggregate-statistic baselines on clean accuracy at small data scale	Measured and not achieved; see 5.5. The claim is robustness and label efficiency
N3	Line-rate production deployment	TRL 4 prototype; data-plane tokenisation is a contemplated embodiment, not built
N4	Per-user or per-identity tracking	Out of scope and adverse to the privacy property
N5	Classifying beyond the first N packets of a flow	Diminishing returns; N between 8 and 128 is the useful band
N6	Supporting nfstream on Windows	Hangs on capture read; replaced by a dpkt extractor

4. Functional Requirements

FR-1 Flow assembly

Observe frames and record only wire length, arrival timestamp and five-tuple. Assemble frames sharing an unordered five-tuple into a bidirectional flow. Designate the endpoint that sent the first observed packet as client, and mark every later packet with its direction relative to that client. Retain only the first N packets. Discard flows below a minimum packet count. **The frame body is never read and may be discarded as soon as the length is known.**

FR-2 Flow-shape tokenisation

For each retained packet derive a size, an inter-arrival time and a direction bit. Clamp size to the MTU and quantise onto S logarithmic bins; clamp inter-arrival time and quantise onto T logarithmic bins. Emit either three factor identifiers for a factorised embedding, or a single composite identifier over an alphabet of $2 \cdot S \cdot T$ tokens plus reserved padding, classification and mask identifiers.

FR-3 Masked-flow-token pre-training

Select a fraction of non-padding positions and replace their embedding with a learned mask vector for the majority of selections, leaving the remainder intact or randomised. Three parallel heads reconstruct the size bin, the time bin and the direction of each selected position, trained with summed cross-entropy on unlabelled flows only.

FR-4 Supervised fine-tuning

Fit a classification head over the flow representation on a labelled set. The encoder may be fully fine-tuned or frozen as a linear probe. Tasks: application identification, and a benign-versus-malicious determination including command-and-control detection.

FR-5 Flow representation pooling

Form the representation by concatenating the encoder output at a prepended classification token with the mean of the outputs at the real packet positions.

FR-6 Evasion-robust training

Before tokenisation, inflate packet sizes by a bounded random fraction with probability p and add non-negative Gaussian jitter to inter-arrival times, drawing fresh randomness on every access so successive epochs present different perturbations of the same flow. The same transform with a fixed seed serves as the evaluation-time attack.

FR-7 Hybrid representation (optional)

Concatenate the learned flow embedding with aggregate flow statistics standardised using training-split statistics only, before classification.

FR-8 Baselines

Maintain RandomForest and XGBoost over aggregate flow statistics as the comparison bar. These are not to be removed when the Transformer wins; both numbers are required.

FR-9 Evaluation

Report accuracy and macro-F1 on a held-out stratified split, a label-efficiency sweep across label budgets, clean-versus-evaded robustness, confusion matrices and an embedding projection.

FR-10 Live demonstration interface

Five tabs: live classification with running accuracy, tokenisation inspector, embedding map, interactive evasion panel with attacker-controlled sliders, and provenance. Demo numbers must match the command-line evaluation exactly.

FR-11 Provenance

Every pipeline run appends a record containing UTC timestamp, git commit, dirty flag, config hash, seed, device and library versions.

FR-12 Synthetic data generator

Generate a dataset whose classes share aggregate statistics and differ only in packet ordering, so the pipeline runs end to end with no downloads and the order-modelling claim can be isolated.

5. Success Metrics (measured, not targeted)

All figures below were produced by this implementation and verified against a clean run. Synthetic results use 7200 flows across six order-defined classes with 1080 held out. Real results use 1797 self-captured flows across five activity classes with 270 held out.

5.1 Does the method model sequence? (G2)

Method	Representation	Accuracy	Macro-F1
RandomForest	Aggregate flow statistics	0.5009	0.4982
XGBoost	Aggregate flow statistics	0.4833	0.4812
CipherFlow	Flow-shape token sequence	0.9944	0.9944

Both statistical baselines sit near chance for six classes. The information they consume does not separate classes defined by ordering. **Met.**

5.2 Label efficiency (G3)

Labels per class	CipherFlow (pre-trained)	Same encoder, from scratch	RF / XGB	Gain
5	0.789	0.365	0.346	+0.424
25	0.892	0.561	0.418	+0.331
100	0.993	0.990	0.457	+0.003
500	0.998	0.999	0.488	-0.001

At five labels per class, pre-training more than doubles the from-scratch macro-F1. The advantage closes by a hundred per class, which is the expected behaviour: pre-training substitutes for labels and stops paying once labels exist. **Met.**

5.3 Evasion robustness on real traffic (G4)

Model	Clean	Under attack	Change	Relative loss
RandomForest on flow statistics	0.6481	0.5333	-0.1148	17.7%
CipherFlow, conventionally trained	0.5519	0.4852	-0.0667	12.1%
CipherFlow, evasion-augmented	0.5370	0.5519	+0.0148	none

The augmented model loses nothing to the attack and overtakes the RandomForest under attack (0.5519 against 0.5333) having started behind it. The ranking of the two methods inverts exactly when an adversary is present. **Met.**

5.4 Pre-training health

Dataset	First epoch	Final epoch	Epochs
Synthetic, order-defined	0.485	0.669	12
Captured real traffic	0.430	0.751	20

Masked-token reconstruction accuracy, not classification accuracy. The rise indicates structure is being learned rather than marginals memorised.

5.5 Result recorded against the project

On clean real traffic the RandomForest beats CipherFlow, 0.6481 against 0.5519. The hybrid head (FR-7) did not close the gap: by macro-F1 on identical splits, statistical features alone scored 0.6597, the embedding alone 0.5296, and the two concatenated 0.5780.

The capture is 1797 flows, which is small, and the sequence model is the more data-hungry of the two, so this ordering is expected at this scale. It is recorded rather than omitted because the claims rest on payload-free operation, label efficiency and robustness under attack, none of which depend on winning clean accuracy on a small residential capture. **Non-goal N2.**

5.6 Engineering metrics

Metric	Value
Model parameters	541,056
Composite token vocabulary	1,028
Flow representation dimension	256
Per-flow retained state	3 small integers per packet, under 100 bytes at N=32
Source lines (package, scripts, tests)	4,674
Python modules	37, all importing cleanly

Metric	Value
VRAM ceiling	6 GB; also runs CPU-only

6. Non-Functional Requirements

#	Requirement	Status
NFR-1	Train inside 6 GB VRAM and run on CPU where no accelerator is present	Met; AMP on, batch tuned, CUDA auto-detected
NFR-2	No stage of the pipeline may read packet payload	Met; enforced by the data path, and surfaced in the tokenisation inspector which reports payload bytes read: 0
NFR-3	Per-flow state must not grow with payload volume	Met; fixed-length token sequence regardless of flow size
NFR-4	Every artifact traceable to a code revision, config hash and seed	Met; <code>artifacts/run_manifest.json</code>
NFR-5	Smoke tests runnable with no real data present	Met; <code>tests/test_smoke.py</code>
NFR-6	Parquet via pyarrow as the only data interchange format	Met; CSV not reintroduced
NFR-7	Demo figures must match command-line evaluation exactly	Met; both score the same stratified test split
NFR-8	Pipeline reproducible in one command	Met; <code>python -m cipherflow.pipeline --quick --fewshot</code>
NFR-9	Lint and tests green on every push	Met; GitHub Actions runs ruff, pytest and an end-to-end smoke

7. System Architecture

7.1 Pipeline

```
pcap -> extract_pcap (dpkt) -> flows.parquet -> FlowTokenizer
      -> masked-flow-token pretrain (unlabelled) -> finetune (small labelled set)
      -> eval / robustness -> serve
```

7.2 Module map

Module id	Responsibility	Depends on
<code>utils</code>	Config loading with <code>--set</code> overrides, seeding, device selection, UTF-8 console	—
<code>data</code>	pcap extraction (dpkt), synthetic generator, parquet I/O	<code>utils</code>
<code>tokenizer</code>	Flow-shape quantisation and tokenisation, the novel element	<code>utils</code>
<code>model</code>	FlowFormer encoder, MLM and classifier heads	<code>utils</code> , <code>tokenizer</code>
<code>pretrain</code>	Masked-flow-token self-supervised training	<code>model</code> , <code>tokenizer</code> , <code>data</code>
<code>finetune</code>	Supervised heads, statistical features, hybrid GBM, baselines	<code>model</code> , <code>tokenizer</code> , <code>data</code>
<code>robustness</code>	Padding and jitter transforms, evasion evaluation	<code>tokenizer</code> , <code>finetune</code>
<code>eval</code>	Metrics, few-shot sweep, embedding projection	<code>finetune</code> , <code>robustness</code>
<code>inference</code>	Single code path from raw flow to class probabilities	<code>model</code> , <code>tokenizer</code>
<code>serve</code>	Streamlit demo, five tabs	<code>inference</code> , <code>eval</code> , <code>robustness</code>
<code>provenance</code>	Run stamping into the manifest	<code>utils</code>
<code>deliverables</code>	Patent figures, results, robustness figure	<code>eval</code> , <code>robustness</code>
<code>pipeline</code>	One-command orchestrator over all of the above	all

Build order: `utils` → `data`, `tokenizer` → `model` → `pretrain`, `finetune` → `robustness`, `eval` → `inference` → `serve`, `deliverables`.

7.3 Key design decisions

Logarithmic quantisation, not linear. Small protocol packets receive finer resolution than bulk transfer packets, and a bounded fractional size increase applied by an evader frequently leaves a packet in the bin it already occupied. Linear binning has neither property. This is why the evasion robustness in 5.3 works at all.

Discrete tokens, not continuous features. A masked reconstruction objective needs a finite alphabet to predict over. Continuous shape features offer nothing to mask and reconstruct, which is precisely why statistical methods cannot be pre-trained.

Concatenated classification-token and masked-mean pooling. The classification token captures flow-level character; the masked mean retains evidence spread across individual packets. The concatenation outperformed either alone.

dpkt over nfstream. nfstream hangs on Windows when reading a capture. `extract_flows.py` is retained only as a non-Windows fallback.

Small model by design. 541k parameters, four layers, width 128. Fits the 6 GB ceiling with room, trains in seconds per epoch, and keeps the contemplated data-plane embodiment plausible.

8. Commands

```
# Environment (conda ml; CUDA wheel first so nothing drags a CPU build over it)
conda activate ml
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install -e ".[dev]"

# Full reproduction, synthetic
python -m cipherflow.pipeline --quick
python -m cipherflow.pipeline --quick --fewshot
python -m cipherflow.pipeline --quick --set model.d_model=64 pretrain.epochs=3

# Individual stages
python -m cipherflow.data.make_synthetic --mode sequential --n-per-class 1200
python -m cipherflow.data.extract_pcap --dir raw/mycapture --out data_out/selfcap.parquet
python -m cipherflow.pretrain.train_mlm --data data_out/selfcap_f12.parquet
python -m cipherflow.finetune.train_clf --pretrained artifacts/pretrained.pt
python -m cipherflow.finetune.train_clf --augment
python -m cipherflow.finetune.baselines
python -m cipherflow.robustness.eval_robustness --ckpt artifacts/classifier_aug.pt
python -m cipherflow.eval.run_fewshot --ks 5 25 100 500
python -m cipherflow.deliverables.real_robustness

# Demo, provenance, deliverables
python -m streamlit run cipherflow/serve/app.py
python -m cipherflow.provenance
python scripts/make_idf.py --pdf
python scripts/capture_demo_shots.py --port 8531

# Quality gates
pytest -q
ruff check cipherflow tests scripts
```

Checkpoint safety. `artifacts/classifier.pt` and `pretrained.pt` are the real-traffic demo models the disclosure quotes. The pipeline writes to those paths by default, so redirect a test run with `--set finetune.ckpt_path=artifacts/scratch/classifier.pt pretrain.ckpt_path=artifacts/scratch/pretrained.pt`.

9. Project Structure

```

cipherflow/
  data/           Flow extraction (extract_pcap.py, dpkt) + synthetic generator
  tokenizer/      Flow-shape tokenisation and quantisation, the novel element
  model/          FlowFormer encoder, MLM and classifier heads
  pretrain/       Masked-flow-token self-supervised training
  finetune/       Supervised heads, baselines, features, hybrid GBM
  robustness/     Padding and jitter transforms, evasion evaluation
  eval/           Metrics, few-shot sweep, embedding projection
  serve/          Streamlit demo, five tabs
  deliverables/   Patent draft, results, figures, this PRD
  configs/        default.yaml, every hyperparameter
  pipeline.py     One-command orchestrator
  inference.py    Single raw-flow-to-probabilities path
  provenance.py   Run stamping
  scripts/        IDF generator, demo screenshot capture, pcap capture
  tests/          test_smoke.py, runnable with no data present
  .github/workflows CI
  raw/ data_out/ artifacts/  Data and outputs, gitignored

```

10. Code Style

Match the surrounding file. Docstrings state what a module is for and how to run it; comments explain why a non-obvious choice was made, not what the line does.

```

def pad_sizes(sizes: np.ndarray, pad_prob: float, pad_max_frac: float, rng, max_size: int = 1500) -> np.
    """Inflate a random subset of packet sizes by up to pad_max_frac (never past MTU)."""
    sizes = sizes.astype(float).copy()
    hit = rng.random(len(sizes)) < pad_prob
    frac = rng.uniform(0.0, pad_max_frac, len(sizes))
    sizes[hit] = np.minimum(sizes[hit] * (1 + frac[hit]), max_size)
    return sizes

```

Conventions: `from __future__ import annotations` at the top; type hints on public functions; snake_case; configuration through `load_config(overrides=...)` rather than constants scattered across modules; ruff with line length 110 selecting `E, F, I, UP, B`. House-style exceptions (`E702` for paired `model.eval()`;

`head.eval()` calls, `E402` for the demo's sys.path bootstrap) are configured once in `pyproject.toml` rather than worked around per file.

11. Testing Strategy

Level	Where	What it covers
Smoke	<code>tests/test_smoke.py</code>	Tokeniser output shapes and bin ranges, forward pass, MLM loss and backward, gradient flow, classifier head shape. Runs in seconds on CPU with no data files
Self-check	<code>cipherflow/provenance.py::demo()</code>	Config hash stability, timestamp format, device string
End-to-end	<code>pipeline --quick --mode easy --n-per-class 60</code> in CI	All eight stages wire together on a tiny budget
Lint	<code>ruff check cipherflow tests scripts</code>	Import order, unused names, modernisation, bug-prone patterns

CI (`.github/workflows/ci.yml`) runs install, lint, unit tests, pipeline smoke and artifact upload on every push and pull request, against CPU torch.

Coverage expectation. This is a research prototype, so there is no percentage target. The requirement is that the smoke test stays runnable without real data, and that the CI pipeline smoke exercises every stage.

12. Boundaries

Always

- Keep the payload-free property intact; any change that reads payload bytes breaks the core claim
- Keep the RandomForest and XGBoost baselines; both numbers are needed
- Parquet via pyarrow for data interchange
- Stamp provenance on every run that produces a quoted number
- Redirect checkpoint paths on a test run so the demo models survive
- Keep the code working on CPU and inside 6 GB VRAM
- Verify a number against a run before putting it in a document

Ask first

- Changing the tokenisation scheme, bin counts or sequence length (invalidates every checkpoint)
- Adding a dependency, naming the conda env and the reason

- Changing the config schema or default hyperparameters
- Retraining or replacing `artifacts/classifier.pt` or `pretrained.pt`
- Any change to the claim set in `patent_draft.md`

Never

- Commit `patent_draft.md`, `CipherFlow_IDF.docx` or `CipherFlow_IDF.pdf`. A public repository is a public disclosure, and India and the EPO apply absolute novelty with no grace period
- Make the repository public before the application has a filing date
- Reintroduce CSV, or nfstream as the default extractor
- Push, open a pull request or force-push without being asked
- Report a metric that no run produced

13. Milestones and Status

Phase	Deliverable	Status
M1	Flow-shape tokeniser and FlowFormer encoder	Complete
M2	Masked-flow-token pre-training	Complete
M3	Fine-tuning, baselines, synthetic headline result	Complete
M4	Evasion transforms and robustness evaluation	Complete
M5	Real traffic capture and extraction (dpkt)	Complete, 1797 flows, 5 classes
M6	Label-efficiency sweep	Complete
M7	Streamlit demo, five tabs	Complete
M8	Provenance, CI, packaging	Complete
M9	Patent draft and invention disclosure form	Complete, 19 claims, TRL 4
M10	Patent-literature search	Complete, Orbit FamPat; see section 15
M11	Public benchmark corpus (ISCXVPN2016)	Not started
M12	Attorney review and filing	Not started

14. Risks and Mitigations

#	Risk	Severity	Mitigation
R1	Public disclosure before filing destroys novelty	Critical	Repository made private; disclosure files gitignored; the 2026-10-04 incident was recorded and remediated the same day
R2	<code>scripts/make_idf.py</code> embeds the full disclosure as string literals	High	Acceptable while private, must not be published. Splitting the content into a gitignored data file is available if needed
R3	Real-traffic clean accuracy below the statistical baseline	Medium	Recorded openly in 5.5; claims rest on robustness and label efficiency instead. Addressed long-term by M11
R4	Real capture is small (1797 flows) and single-environment	Medium	Synthetic testbed isolates the mechanism; M11 extends to a public corpus
R5	Patent search returned zero on the narrow query	Medium	A zero alone is weak evidence. The closest prior art is academic (ET-BERT, FlowPic, Deep Packet, nPrint) and would never appear in a patent database, so manual literature differentiation carries the argument
R6	Section 3(k) of the Indian Patents Act excludes computer programmes per se	High	Claims anchored to a network monitoring apparatus with recited technical effect; section 7 of the patent draft sets out seven effects
R7	A pipeline run silently overwrites the demo checkpoints	Medium	Fixed: checkpoint paths read from config, the few-shot sweep writes to scratch, and checkpoints record their training dataset
R8	Documents drift from measured results	Medium	The IDF is generated by script, results are regenerated from runs, and every number is verified against a manifest

15. Prior Art Position

Six references reviewed, each differentiated in `patent_draft.md` section 3. The closest is **ET-BERT** (Lin et al., WWW 2022), which applies masked pre-training to encrypted traffic but tokenises payload byte n-grams, so it depends on observable payload and is defeated by full encryption. **FlowPic** uses shape but aggregates it into a histogram, destroying packet order, and is fully supervised. **Deep Packet** and **nPrint** are payload- and header-driven respectively, both supervised.

A patent-literature search in Orbit (FamPat) across CPC classes H04L63/1408, H04L63/1425, H04L43/02 and G06N20/00 returned no documents combining payload-free flow-shape features with masked self-supervised pre-training, run as a narrowing ladder to confirm the query responded to each added condition.

The inventive step is the combination, not any single element. Masked pre-training is known from language modelling. Shape features are known to statistical classifiers. Adversarial augmentation is known generally. What does not appear anywhere is a masked objective defined over a payload-free quantised shape alphabet, and that is the bridge making self-supervision available at all when no bytes can be read.

16. Open Questions

1. Does the clean-accuracy gap in 5.5 close on a larger public corpus, or is it intrinsic to the representation at this model scale?
 2. Should the hybrid head (FR-7) be retained given it underperformed statistical features alone? It is kept because behaviour may differ at scale, but that is untested.
 3. Is a provisional filing worth making now to secure the date while M11 runs, or should the complete specification wait for public-corpus results?
 4. Does the claim set survive attorney review, particularly claim 10's reference dependency on claim 1 and the Section 3(k) framing?
 5. Which CPC class should be primary on filing?
-

17. Out of Scope and Future Work

- Data-plane tokenisation on a programmable switch or SmartNIC, contemplated in the specification as an embodiment but not built
- Extending the direction bit to a role encoding (client, server, relay)
- Learned or quantile-fitted bin edges in place of fixed logarithmic binning
- A contrastive objective alongside masked reconstruction
- Replacing the Transformer with a state-space or dilated-convolutional sequence model
- Evaluation against an adaptive adversary who knows the defence is in place